



**Institute of
Emerging
Technologies**



**Institute of
Emerging
Technologies**

COMPILER CONSTRUCTION

MA'AM MUNIZA

**NAME: MUREED
ABBAS**

CLASS: BSCS V

SECTION: A

CMS ID : 023-24-0439

LAB : Four

Task 1 — Rename files with a for loop

Task

Use a for loop to rename existing files in a folder. For example, file1.txt, file2.txt, file3.txt should become new-file1.txt, new-file2.txt, new-file3.txt.

Design note

The for loop iterates over the shell's own wildcard expansion file*.txt, so it automatically picks up however many matching files exist — not just three. The -f test guards against the loop variable ever being a non-file (e.g. if a directory happened to match the pattern).

Solution — tasks/task1_rename.sh

```
#!/bin/bash

# Task 1: Rename file1.txt, file2.txt, file3.txt -> new-file1.txt, new-file2.txt, new-
file3.txt

for file in file*.txt
do
    if [ -f "$file" ]; then
        newname="new-$file"
        mv "$file" "$newname"
        echo "Renamed: $file -> $newname"
    fi
done
```

Run

```
user@ubuntu:~$ ls
file1.txt  file2.txt  file3.txt

user@ubuntu:~$ ./task1_rename.sh
```

Verified output

```
Renamed: file1.txt -> new-file1.txt
Renamed: file2.txt -> new-file2.txt
Renamed: file3.txt -> new-file3.txt
```

Folder after running

```
user@ubuntu:~$ ls
new-file1.txt  new-file2.txt  new-file3.txt
```

Task 2 — Bio-data into an array

Task

Get bio-data from the user and save it to an array, then print the bio-data.

Design note

Five separate read -p prompts collect each field, which are then packed into one array (biodata) alongside a parallel labels array. Looping over "\${!biodata[@]}" (the array's indices) lets the same loop print every "Label: value" pair without hard-coding five separate echo lines.

Solution — tasks/task2_biodata.sh

```
#!/bin/bash

# Task 2: Get Bio-Data from the user, save it to an array, then print it.

echo "----- Bio-Data Entry -----"
read -p "Enter your Name: " name
read -p "Enter your Father Name: " fname
read -p "Enter your Age: " age
read -p "Enter your City: " city
read -p "Enter your Occupation: " occupation

# Save all fields into a single array
biodata=("$name" "$fname" "$age" "$city" "$occupation")
labels=("Name" "Father Name" "Age" "City" "Occupation")

echo
echo "----- Your Bio-Data -----"
for i in "${!biodata[@]}"
do
    echo "${labels[$i]}: ${biodata[$i]}"
done
```

Run (input piped for automated testing)

```
user@ubuntu:~$ printf "Ahmed Ali\nZafar Iqbal\n22\nKarachi\nStudent\n" |  
./task2_biodata.sh
```

Verified output

```
----- Bio-Data Entry -----  
  
----- Your Bio-Data -----  
Name: Ahmed Ali  
Father Name: Zafar Iqbal  
Age: 22  
City: Karachi  
Occupation: Student
```

Task 3 — 5 files, 5 permission sets, all file conditions

Task

Create 5 files with different permissions and apply all the file-conditional tests (-d, -f, -r, -s, -w, -x) to each.

Design note

Five files are created with chmod 400, 200, 100, 777 and 000 respectively — read-only, write-only, execute-only, fully open, and no permissions at all — and fileD is additionally given nonzero size so the -s test has something to detect. All six file conditionals from the manual's table are then run against every file in one loop. Note: if this script is run as root, -r and -w will always report TRUE regardless of the chmod bits, because root bypasses standard Unix permission checks; the output below was captured running as a normal (non-root) user so it reflects the real permission bits.

Solution — tasks/task3_file_conditions.sh

```
#!/bin/bash  
  
# Task 3: Create 5 files with different permissions and apply all file conditions.
```

```
# Clean any previous run and create a fresh sandbox
rm -rf fcheck && mkdir fcheck && cd fcheck

# 1) readable only, 2) writable only, 3) executable only,
# 4) all permissions, 5) no permissions at all
touch fileA fileB fileC fileD fileE

chmod 400 fileA # r----- readable only
chmod 200 fileB # -w----- writable only
chmod 100 fileC # --x----- executable only
chmod 777 fileD # rwxrwxrwx fully open
chmod 000 fileE # ----- no permissions

echo "hello world" >> fileD # give fileD nonzero size (it is the only writable+readable
one)

files=(fileA fileB fileC fileD fileE)

for f in "${files[@]}"
do
    echo "==== $f (perms: $(stat -c '%A' "$f")) ====="
    [ -d "$f" ] && echo " -d : TRUE (is a directory)" || echo " -d : FALSE (not a
directory)"
    [ -f "$f" ] && echo " -f : TRUE (is a regular file)" || echo " -f : FALSE (not a
regular file)"
    [ -r "$f" ] && echo " -r : TRUE (is readable)" || echo " -r : FALSE (not
readable)"
    [ -s "$f" ] && echo " -s : TRUE (has nonzero size)" || echo " -s : FALSE (empty /
zero size)"
    [ -w "$f" ] && echo " -w : TRUE (is writable)" || echo " -w : FALSE (not
writable)"
    [ -x "$f" ] && echo " -x : TRUE (is executable)" || echo " -x : FALSE (not
executable)"
    echo
```

```
done
```

```
cd ..
```

Run (as a normal user)

```
user@ubuntu:~$ ./task3_file_conditions.sh
```

Verified output

```
===== fileA (perms: -r-----) =====
```

```
-d : FALSE (not a directory)
-f : TRUE  (is a regular file)
-r : TRUE  (is readable)
-s : FALSE (empty / zero size)
-w : FALSE (not writable)
-x : FALSE (not executable)
```

```
===== fileB (perms: --w-----) =====
```

```
-d : FALSE (not a directory)
-f : TRUE  (is a regular file)
-r : FALSE (not readable)
-s : FALSE (empty / zero size)
-w : TRUE  (is writable)
-x : FALSE (not executable)
```

```
===== fileC (perms: ---x-----) =====
```

```
-d : FALSE (not a directory)
-f : TRUE  (is a regular file)
-r : FALSE (not readable)
-s : FALSE (empty / zero size)
-w : FALSE (not writable)
-x : TRUE  (is executable)
```

```
==== fileD (perms: -rwxrwxrwx) ====
-d : FALSE (not a directory)
-f : TRUE  (is a regular file)
-r : TRUE  (is readable)
-s : TRUE  (has nonzero size)
-w : TRUE  (is writable)
-x : TRUE  (is executable)

==== fileE (perms: -----) ====
-d : FALSE (not a directory)
-f : TRUE  (is a regular file)
-r : FALSE (not readable)
-s : FALSE (empty / zero size)
-w : FALSE (not writable)
-x : FALSE (not executable)
```

Task 4 — Number to Roman numeral (1–10) using case

Task

Prompt the user for a number from 1 through 10 and use case to display its Roman numeral. Outside that range, display an error message.

Solution — tasks/task4_roman.sh

```
#!/bin/bash

# Task 4: Convert a number (1-10) to its Roman numeral using case.

read -p "Enter a number between 1 and 10: " num

case "$num" in
    1) echo "Roman numeral: I"    ;;
```

```
2) echo "Roman numeral: II" ;;
3) echo "Roman numeral: III" ;;
4) echo "Roman numeral: IV" ;;
5) echo "Roman numeral: V" ;;
6) echo "Roman numeral: VI" ;;
7) echo "Roman numeral: VII" ;;
8) echo "Roman numeral: VIII";;
9) echo "Roman numeral: IX" ;;
10) echo "Roman numeral: X" ;;
*) echo "Error: $num is outside the range 1-10" ;;
esac
```

Run & verified output

```
user@ubuntu:~$ echo "7" | ./task4_roman.sh
Roman numeral: VII

user@ubuntu:~$ echo "10" | ./task4_roman.sh
Roman numeral: X

user@ubuntu:~$ echo "15" | ./task4_roman.sh
Error: 15 is outside the range 1-10
```

Task 5 — Numbers into an array until "no"

Task

Take number input from the user and save it to an array until the user enters "no". Print the length of the array.

Design note

An infinite while true loop reads one value per iteration; typing "no" hits break before that value is appended, so "no" itself is never stored. Every other value is appended with `numbers+=("$value")`, and `${#numbers[@]}` gives the final count.

Solution — `tasks/task5_numarray.sh`


```
#!/bin/bash

# Task 5: Take numbers from the user and store them in an array until the
# user types "no". Then print the length of the array.

numbers=()

while true
do
    read -p "Enter a number (or 'no' to stop): " value
    if [ "$value" == "no" ]; then
        break
    fi
    numbers+=("$value")
done

echo
echo "Numbers entered: ${numbers[@]}"
echo "Length of the array: ${#numbers[@]}"
```

Run (input piped for automated testing)

```
user@ubuntu:~$ printf "5\n12\n7\n21\nno\n" | ./task5_numarray.sh
```

Verified output

```
Numbers entered: 5 12 7 21
Length of the array: 4
```

Task 6 — Find files by extension (function)

Task

Ask the user for a file extension, then write a function that crawls the current directory and displays all files with that extension.

Design note

find_by_extension() takes the extension as its own local parameter (\$1), so it does not depend on any global variable and could be reused for other extensions in the same script. It loops over the wildcard expansion *.\$ext, checks -f to skip non-matches (the literal, unexpanded pattern if nothing matches), and reports explicitly when nothing was found.

Solution — tasks/task6_findext.sh

```
#!/bin/bash

# Task 6: Ask the user for a file extension, then use a function to crawl
# through the current directory and display all files with that extension.

find_by_extension() {
    local ext="$1"
    local found=0
    for f in *."$ext"
    do
        if [ -f "$f" ]; then
            echo "$f"
            found=1
        fi
    done
    if [ $found -eq 0 ]; then
        echo "No files with extension .$ext were found in $(pwd)"
    fi
}

read -p "Enter a file extension to search for (without the dot, e.g. txt): " extension

echo
echo "Files with extension .$extension in $(pwd):"
find_by_extension "$extension"
```

Run

```
user@ubuntu:~$ ls
```

```
a.txt b.txt image.png notes.md report.txt
user@ubuntu:~$ echo "txt" | ./task6_findext.sh
```

Verified output

```
Files with extension .txt in /home/user/test6:
a.txt
b.txt
report.txt
```

Task 7 — Read a text file line by line

Task

Write a script that reads a text file line by line and displays it on the screen.

Design note

while IFS= read -r line ... done < "\$filename" is the standard, safe idiom for line-by-line reading: IFS= preserves leading/trailing whitespace on each line, and -r stops backslashes from being interpreted as escape characters, so the file is displayed exactly as written.

Solution — tasks/task7_readfile.sh

```
#!/bin/bash

# Task 7: Read a text file line by line and display it on the screen.

read -p "Enter the filename to read: " filename

if [ ! -f "$filename" ]; then
    echo "Error: '$filename' does not exist or is not a regular file."
    exit 1
fi

line_num=1
while IFS= read -r line
do
    echo "$line_num: $line"
done
```

```
    line_num=$((line_num + 1))  
done < "$filename"
```

Run

```
user@ubuntu:~$ cat notes.txt  
Operating Systems Lab  
Week 4: Shell Scripting  
Arrays, loops, and functions  
user@ubuntu:~$ echo "notes.txt" | ./task7_readfile.sh
```

Verified output

```
1: Operating Systems Lab  
2: Week 4: Shell Scripting  
3: Arrays, loops, and functions
```

Task 8 — Print 1 to 10 with WHILE and UNTIL

Task

Write a script to print 1 to 10 using both WHILE and UNTIL.

Solution — tasks/task8_while_until.sh

```
#!/bin/bash  
  
# Task 8: Print 1 to 10 using both WHILE and UNTIL loops.  
  
echo "Using WHILE loop:"  
i=1  
while [ $i -le 10 ]  
do  
    echo -n "$i "  
    i=$((i + 1))  
done
```

```
echo
echo

echo "Using UNTIL loop:"
j=1
until [ $j -gt 10 ]
do
    echo -n "$j "
    j=$((j + 1))
done
echo
```

Run & verified output

```
user@ubuntu:~$ ./task8_while_until.sh

Using WHILE loop:
1 2 3 4 5 6 7 8 9 10

Using UNTIL loop:
1 2 3 4 5 6 7 8 9 10
```

Task 9 — Login with 3 tries and a hidden password

Task

Set two variables named username and password. Take input from the user for username and password; if they match, print "Access granted." The user gets three tries, and the password must be invisible while being typed.

Design note

`read -s` suppresses terminal echo while the password is typed, exactly like a real login prompt (an explicit `echo` afterwards restores the newline that `-s` also swallows). The while loop bounds the attempt counter at 3; a successful match sets a flag and `break`'s out immediately, and if the loop runs out without a match, a final "Access denied" message is printed once, after the loop.

Solution — tasks/task9_login.sh

```
#!/bin/bash

# Task 9: Username/password check with 3 tries; password input hidden.

username="admin"
password="secret123"

max_tries=3
attempt=1
access=0

while [ $attempt -le $max_tries ]
do
    echo "Attempt $attempt of $max_tries"
    read -p "Username: " input_user
    read -s -p "Password: " input_pass
    echo  # newline after hidden password input

    if [ "$input_user" == "$username" ] && [ "$input_pass" == "$password" ]; then
        echo "Access granted"
        access=1
        break
    else
        echo "Invalid username or password"
    fi

    attempt=$((attempt + 1))
done

if [ $access -eq 0 ]; then
    echo "Access denied: maximum attempts exceeded"
```

```
fi
```

Run 1 — correct on first try

```
user@ubuntu:~$ printf "admin\nsecret123\n" | ./task9_login.sh
```

```
Attempt 1 of 3
```

```
Access granted
```

Run 2 — wrong twice, then correct on the 3rd try

```
user@ubuntu:~$ printf "admin\nwrong1\nadmin\nwrong2\nadmin\nsecret123\n" |  
./task9_login.sh
```

```
Attempt 1 of 3
```

```
Invalid username or password
```

```
Attempt 2 of 3
```

```
Invalid username or password
```

```
Attempt 3 of 3
```

```
Access granted
```

Run 3 — fails all 3 tries

```
user@ubuntu:~$ printf "admin\nbad1\nadmin\nbad2\nadmin\nbad3\n" | ./task9_login.sh
```

```
Attempt 1 of 3
```

```
Invalid username or password
```

```
Attempt 2 of 3
```

```
Invalid username or password
```

```
Attempt 3 of 3
```

```
Invalid username or password
```

```
Access denied: maximum attempts exceeded
```

How to Use These Files

All 9 scripts are provided separately in OS_Lab4_Solutions_Source.zip, inside a tasks/ folder. Each was written and tested with GNU bash 5.2 on Ubuntu 24.04.

General recipe

```
user@ubuntu:~$ chmod +x task<N>_<name>.sh  
user@ubuntu:~$ ./task<N>_<name>.sh
```

Every script uses read (or read -p / read -s) for input exactly as a user would type it interactively — just run it directly in your terminal and answer the prompts as they appear.



Institute of
Emerging
Technologies